

Esercitazione di Programmazione 2

Una scuola vuole mantenere una lista degli studenti iscritti ad un corso ordinata automaticamente in base al voto finale ottenuto. Ogni studente è caratterizzato da nome, matricola e voto finale.

Implementare una classe `Studente` contenente gli attributi privati `nome`, `matricola` e `voto`. La classe deve prevedere un costruttore, i metodi `getter` e l'overload degli operatori `<`, `>` e `==` in modo che il confronto tra oggetti `Studente` avvenga rispetto al voto. Implementare inoltre l'overload dell'operatore `<<` per consentire la stampa degli oggetti.

Successivamente implementare una lista concatenata ordinata generica utilizzando i template. Definire una struttura nodo contenente un valore generico e il puntatore al nodo successivo. Creare una classe `OrderedList<T>` che permetta di inserire elementi mantenendo automaticamente l'ordinamento crescente della lista.

La lista deve inoltre fornire un metodo `removeBetween(T min, T max)` che elimini tutti gli elementi compresi tra due valori rispetto all'ordinamento definito dagli operatori overloaded. Implementare anche un metodo `print()` per stampare il contenuto della lista.

Nel main chiedere all'utente quanti studenti desidera inserire. Per ogni studente leggere nome, matricola e voto, creare l'oggetto `Studente` e inserirlo nella lista ordinata. Terminato l'inserimento, stampare la lista completa.

Successivamente chiedere all'utente un voto minimo e un voto massimo ed eliminare dalla lista tutti gli studenti con voto compreso nell'intervallo specificato. Infine stampare nuovamente la lista aggiornata dopo le eliminazioni.

Non è consentito utilizzare strutture dati STL come `std::list`. La lista deve essere implementata manualmente tramite puntatori e gestione dinamica della memoria.